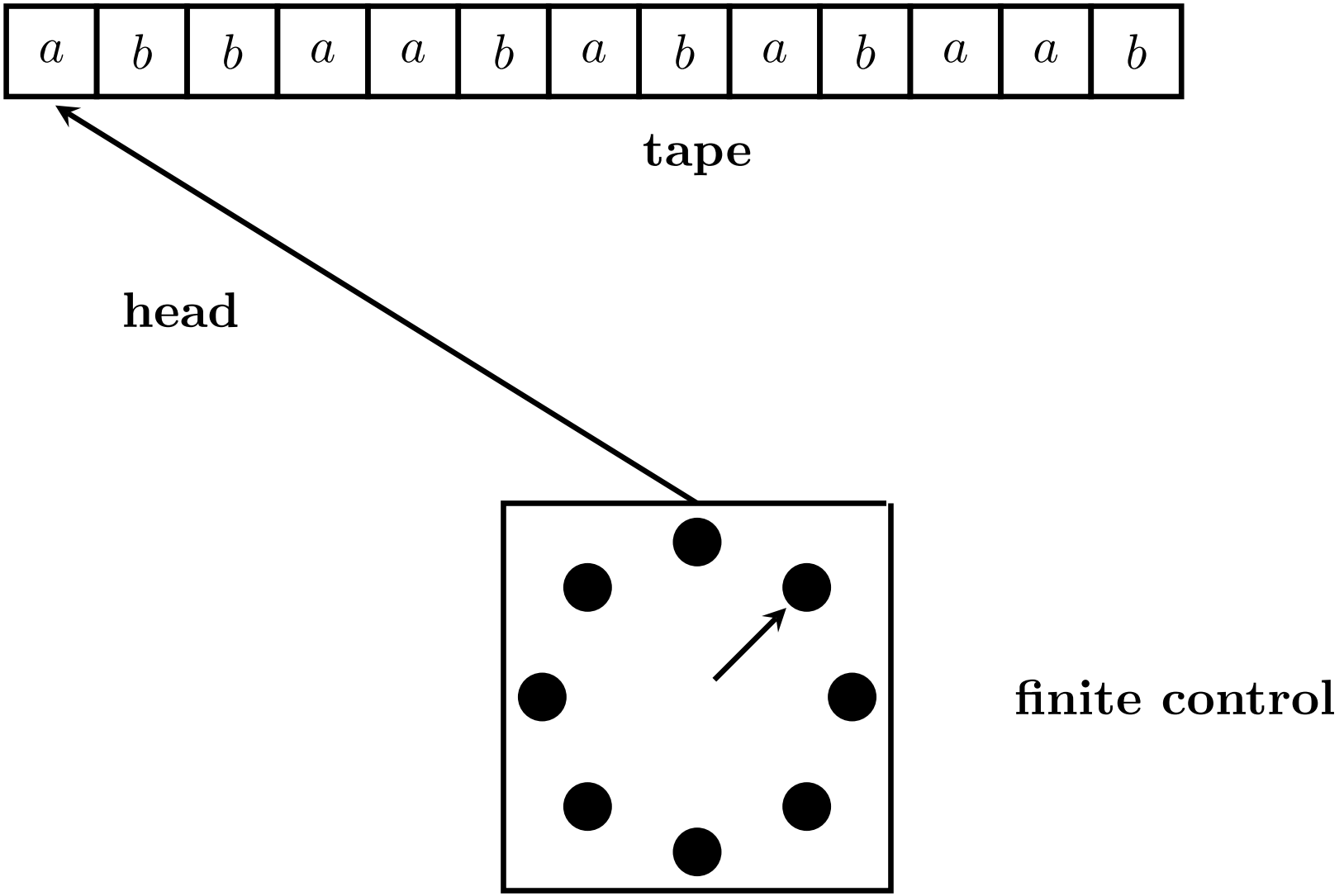
Finite Automata: Deterministic (DFA) and Nondeterministic (NFA

Birzhan Kalmurzayev

Ac. year 2025-2026

Intuition for DFA

- A **deterministic finite automation (DFA)** is a mechanical machine which reads an input sting one letter at a time and then, after the input is completely read, decides to accept or to reject the input.
- A DFA consists of three parts:
  - a **tape** which used to store the input data;
  - a **tape head** which scans one cell of the tape the information to the finite control, and then moves to the next cell to the right;
  - a **finite control** has a finite number of states. At the beginning of a move, the control is in one of the states. Then it determines, from the current state and the symbol read by the tape head, how the state is changed to to a new state.


a b b a a b a b a b a a b
tape
head
finite control

Change of states

The change of state at each move is governed by a **state transition function** $\delta : Q \times \Sigma \to Q$. At each move, if the finite control is currently at state $q \in Q$ and the symbol read by the tape head is $a \in \Sigma$, then the finite control changes to the new state $q = \delta(q, a)$.

Change of states

The change of state at each move is governed by a **state transition function** $\delta : Q \times \Sigma \to Q$. At each move, if the finite control is currently at state $q \in Q$ and the symbol read by the tape head is $a \in \Sigma$, then the finite control changes to the new state $q = \delta(q, a)$.

We also specify two special kinds of states:

- an **initial state** – it is the state which the DFA begins to work,
- a set of **finite states** at which the DFA certifies that th input is in the language.

Formal definition

DFA is a quintuple $M = (Q, \Sigma, \delta, q_0, F)$, where

- Q is the set of states in the finite control;
- Σ is an alphabet for strings in the tape;
- δ is well defined function from $Q \times \Sigma$ to Q;
- q_0 is the initial state;
- F is the set of finite states.

Formal definition

DFA is a quintuple $M = (Q, \Sigma, \delta, q_0, F)$, where

- Q is the set of states in the finite control;
- Σ is an alphabet for strings in the tape;
- δ is well defined function from $Q \times \Sigma$ to Q;
- q_0 is the initial state;
- F is the set of finite states.

In the definition the notion well defined mean that

- function is defined for any input;
- for any input pair there is unique output.

How does DFA work?

Initially, the DFA M is in the initial state, the tape holds the input string, and the tape head scans the leftmost cell of the tape.

How does DFA work?

Initially, the DFA M is in the initial state, the tape holds the input string, and the tape head scans the leftmost cell of the tape.

Then, the DFA M operates, move by move, according to the transition function δ.

How does DFA work?

Initially, the DFA M is in the initial state, the tape holds the input string, and the tape head scans the leftmost cell of the tape.

Then, the DFA M operates, move by move, according to the transition function δ.

The DFA M halts after it reads the symbol in the rightmost cell of the tape and its tape head moves off the tape.

How does DFA work?

Initially, the DFA M is in the initial state, the tape holds the input string, and the tape head scans the leftmost cell of the tape.

Then, the DFA M operates, move by move, according to the transition function δ.

The DFA M halts after it reads the symbol in the rightmost cell of the tape and its tape head moves off the tape.

When DFA halts, it **accepts** the input string if it halts in one of the final states F, otherwise the input string is **rejected**.

How does DFA work?

Initially, the DFA M is in the initial state, the tape holds the input string, and the tape head scans the leftmost cell of the tape.

Then, the DFA M operates, move by move, according to the transition function δ.

The DFA M halts after it reads the symbol in the rightmost cell of the tape and its tape head moves off the tape.

When DFA halts, it **accepts** the input string if it halts in one of the final states F, otherwise the input string is **rejected**.

The set of all strings accepted by a DFA M is denoted by $L(M)$. We also say that the language $L(M)$ is **accepted** by M.

DFA as a labeled digraph

The transition diagram of a DFA is an alternative way to represent the DFA. For $M = (Q, \Sigma, \delta, q_0, F)$, the transition diagram of M is a labeled digraph (V, E) satisfying

- $V = Q$;
- E consists of all edges (p, q) with labed $a \in \Sigma$ iff $\delta(p, a) = q$.

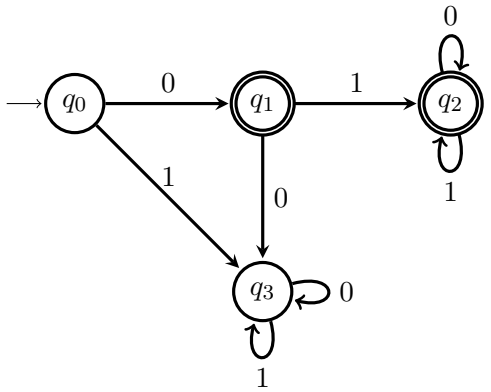
In addition, the initial state of the DFA is pointed to by an arrow withourt a tring vertex, and every final state is denoted by double circles.

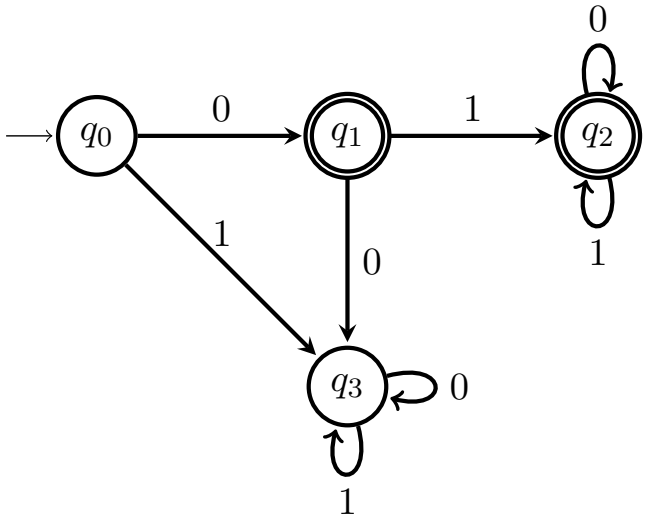
Example

Consider a DFA $M = (Q, \Sigma, \delta, q_0, F)$ where $Q = \{q_0, q_1, q_2, q_3\}$, $\Sigma = \{0, 1\}$, $F = \{q_1, q_2\}$ and δ us the function defined by the following table:

δ	0	1
q_0	q_1	q_3
q_1	q_2	q_3
q_2	q_2	q_2
q_3	q_3	q_3

Then, the transition diagram of M is as the following graph

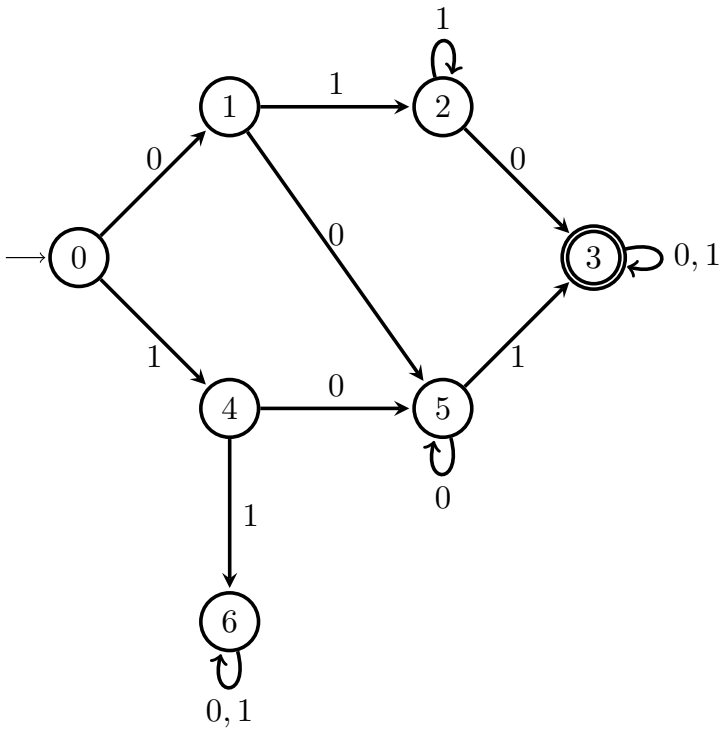




Consider the DFA given as above. Determine weather M accepts the following strings:

- ε;
- 000;
- 111;
- 010;

What is the language $L(M)$ accepted by the DFA.



Example. What is the language $L(M)$ accepted by the DFA M of the figure left (Write its regular expression)?

Theorem

If a language L is accepted by a DFA, then L is regular language.

Regular expression for the L can be form sum of regular expressions for finite states.

Theorem

If a language L is accepted by a DFA, then L is regular language.

Regular expression for the L can be form sum of regular expressions for finite states.

Theorem

If a language L is accepted by a DFA $M_0 = (Q, \Sigma, \delta, q_0, F)$, then the language $\overline{L}$ is accepted by a DFA $M_1 = (Q, \Sigma, \delta, q_0, Q \setminus F)$.

Examples

Construct DFA for which accepted the following languages:

- the set of all binary strings beginning with prefix 101;

Examples

Construct DFA for which accepted the following languages:

- the set of all binary strings beginning with prefix 101;
- the set of all binary strings having a substring 101;

Examples

Construct DFA for which accepted the following languages:

- the set of all binary strings beginning with prefix 101;
- the set of all binary strings having a substring 101;
- the set of all binary strings ending with 101;

Examples

Construct DFA for which accepted the following languages:

- the set of all binary strings beginning with prefix 101;
- the set of all binary strings having a substring 101;
- the set of all binary strings ending with 101;
- the set of all binary expansions of positive integers which are congruent to zero modulo 5;

Examples

Construct DFA for which accepted the following languages:

- the set of all binary strings beginning with prefix 101;
- the set of all binary strings having a substring 101;
- the set of all binary strings ending with 101;
- the set of all binary expansions of positive integers which are congruent to zero modulo 5;
- the set of all binary strings having a substring 101 or ending with 11;

Examples

Construct DFA for which accepted the following languages:

- the set of all binary strings beginning with prefix 101;
- the set of all binary strings having a substring 101;
- the set of all binary strings ending with 101;
- the set of all binary expansions of positive integers which are congruent to zero modulo 5;
- the set of all binary strings having a substring 101 or ending with 11;
- the set of all binary strings having a substring 101 and ending with 11;

Examples

Construct DFA for which accepted the following languages:

- the set of all binary strings beginning with prefix 101;
- the set of all binary strings having a substring 101;
- the set of all binary strings ending with 101;
- the set of all binary expansions of positive integers which are congruent to zero modulo 5;
- the set of all binary strings having a substring 101 or ending with 11;
- the set of all binary strings having a substring 101 and ending with 11;
- the set of all binary strings in which every block of four consecutive symbols contains a substring 01.

Nondeterministic Finite Automata (NFA)

Definition

A NFA $M = (Q, \Sigma, \delta, q_0, F)$ is defined in the same way as a DFA exept that in NFA multiple-state transformation and ε-transformations are allowed, and definition of accepting are different (we will say about it)

Therefore, the transition function δ is, formally, a function of the form

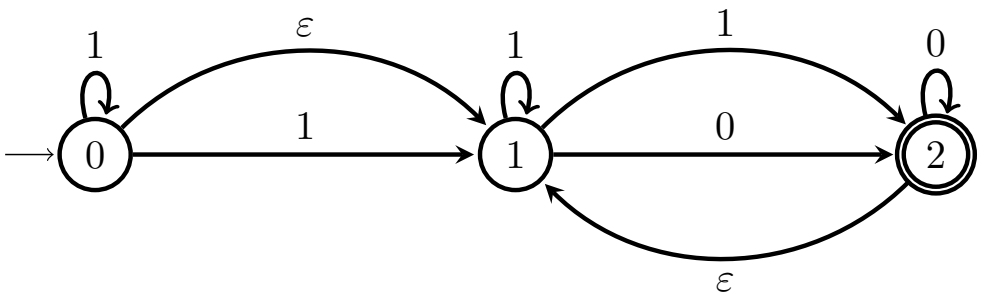
$$\delta : Q \times (\Sigma \cup \varepsilon) \to 2^Q,$$

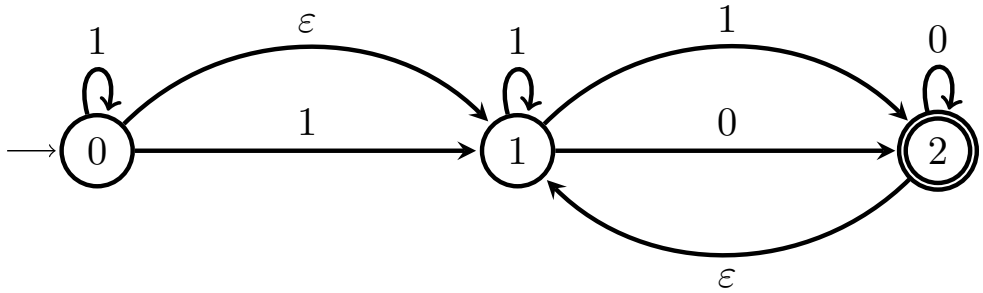
where 2^Q denotes the collection of all subsets of Q.

The following is an example of a transition function of an NFA:

δ	0	1	ε
q_0	$\emptyset$	$\{q_0, q_1\}$	$\{q_1\}$
q_1	$\{q_2\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_2\}$	$\emptyset$	$\{q_1\}$

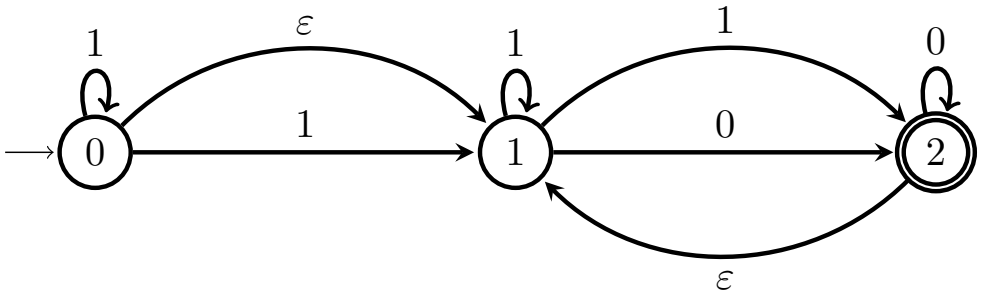
The NFA's, like DFA's, can also be represented by transition diagram.



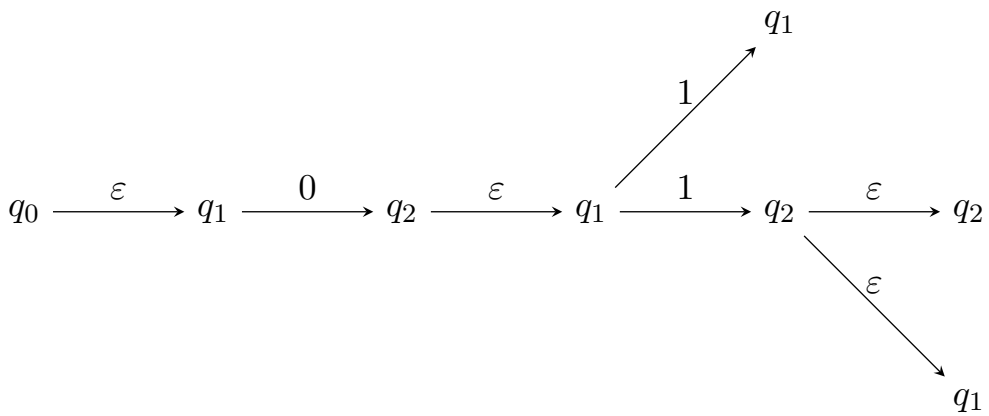


On an input x, an NFA may have more than one computation path. For example, for NFA M shown in figure above, the input 01 has the following three computation paths:

$$q_0 \xrightarrow{\varepsilon} q_1 \xrightarrow{0} q_1 \xrightarrow{\varepsilon} q_1 \xrightarrow{1} q_1,$$
$$q_0 \xrightarrow{\varepsilon} q_1 \xrightarrow{0} q_2 \xrightarrow{\varepsilon} q_1 \xrightarrow{1} q_2,$$
$$q_0 \xrightarrow{\varepsilon} q_1 \xrightarrow{0} q_2 \xrightarrow{\varepsilon} q_1 \xrightarrow{1} q_2 \xrightarrow{\varepsilon} q_1.$$



On an input x, an NFA may have more than one computation path. For example, for NFA M shown in figure above, the input 01 has the following three computation paths (we can merge these paths into one directed tree):



How do we define the notion of an NFA accepting an input? We say that the NFA accepts an input x uf at least one computation path on x leads to a final state. Fir instance, in the last example, since the second computation path ends at state $q_2 \in F$ after the NFA reads the input 01, the string 01 is accepted by the NFA.

How do we define the notion of an NFA accepting an input? We say that the NFA accepts an input x uf at least one computation path on x leads to a final state. Fir instance, in the last example, since the second computation path ends at state $q_2 \in F$ after the NFA reads the input 01, the string 01 is accepted by the NFA.

For instance, graph representation for regular expressions is a graph for NFA automata, i.e.

Theorem

If L is regular language, then there is NFA which accepted L.

Examples

Find NFA's which accepted the following languages:

- the set of all binary stings having substring 010;

Examples

Find NFA's which accepted the following languages:

- the set of all binary stings having substring 010;
- the set of all binary strings beginning with 010 or ending 110;

Examples

Find NFA's which accepted the following languages:

- the set of all binary stings having substring 010;
- the set of all binary strings beginning with 010 or ending 110;
- the set of all binary strings with at least two occurrences of substring 01 and ends with 11;

Examples

Find NFA's which accepted the following languages:

- the set of all binary stings having substring 010;
- the set of all binary strings beginning with 010 or ending 110;
- the set of all binary strings with at least two occurrences of substring 01 and ends with 11;
- the set of all binary strings of form 0^n10^m where $n = m(\mod 5)$.